🪀 Shadow DOM in Test Automation – Handling it with Selenium & Playwright

When working with modern web applications, you might encounter **Shadow DOM** — a powerful browser feature that encapsulates part of the DOM, preventing direct access from outside scripts. While this improves component isolation and styling, it can be tricky for **test automation engineers**.

🔍 What is Shadow DOM?

- Shadow DOM is a *separate DOM tree* attached to an element, hidden from the main document DOM.
- Standard selectors like document.querySelector() won't work directly inside it.
- Example: Many modern UI frameworks (e.g., Web Components, LitElement) use Shadow DOM.

⚡ Handling Shadow DOM in Selenium

Selenium does not have built-in Shadow DOM support. You need to rely on **JavaScript execution** to pierce into shadow roots.

Example in Java (Selenium):

```
WebElement shadowHost = driver.findElement(By.cssSelector("my-component"));

JavascriptExecutor js = (JavascriptExecutor) driver;
```

// Access shadow root

```
WebElement shadowRoot = (WebElement) js.executeScript(
   "return arguments[0].shadowRoot", shadowHost);
```

// Find element inside shadow DOM

```
WebElement innerButton = shadowRoot.findElement(By.cssSelector("button.submit"));

innerButton.click();
```

👉 **Limitation: You must repeatedly fetch nested shadow roots if components are deeply nested.**

⚡ Handling Shadow DOM in Playwright

Playwright makes this easier since it has **built-in piercing selectors**.

Example with CSS piercing (>>>):

```
await page.locator('my-component >>> button.submit').click();
```

Example with Locators (getByRole, getByText, etc.):
Playwright's locator API automatically handles Shadow DOM elements.

```
// Button inside shadow root by role
await page.getByRole('button', { name: 'Submit' }).click();

// Input inside shadow DOM by placeholder
await page.getByPlaceholder('Enter your name').fill('Varsha');

// Link inside shadow DOM by text
await page.getByText('Learn More').click();
```

Example in Python (Playwright):

```
# Using role
await page.get_by_role("button", name="Submit").click()

# Using label
await page.get_by_label("Username").fill("varsha123")

# Using text
await page.get_by_text("Learn More").click()
```

👉 Advantage: No need for custom JavaScript execution. Locators in Playwright **auto-pierce Shadow DOM** as long as the shadow root is **open**.

📊 Selenium vs Playwright – Shadow DOM Handling

Feature	Selenium □	Playwright ⚡
Native Shadow DOM Support	❌ No (needs JS execution)	✅ Yes (built-in)
Selector Style	Manual via JS (shadowRoot.querySelector)	CSS Piercing (>>>), Locator API
Ease of Use	Harder – extra JS needed	Easier – seamless locators
Locator Strategies	Limited (CSS, XPath)	Rich (getByRole, getByText, getByLabel, etc.)
Nested Shadow Roots	Must be handled manually	Automatically supported
Closed Shadow DOM	❌ Not accessible	❌ Not accessible (tool limitation)

🚀 Key Takeaways

- **Selenium** → Requires custom JavaScript to traverse shadow roots.
- **Playwright** → Supports Shadow DOM natively with:
  - >>> CSS piercing selectors
  - getByRole, getByText, getByLabel, getByPlaceholder, etc.
- Both tools cannot access **closed Shadow DOM**.

💡 **Pro Tip:** If you're designing web components, provide **test IDs (data-testid)** inside the Shadow DOM for more stable and readable tests.